



# PRIVOURNAL

## Development Logs

---

*Devlogs #1 — #13 · The complete journey of building a private, offline, command-line journal encryptor from scratch.*

@Manonit

## DEVLOG #1

# The Beginning

*Session time: 31m 25s*

---

Today wasn't about building features — but about getting everything ready.

I finished setting up the GitHub repo (took me an hour lol), cleaned things up, and wrote the first version of the README.

It might not seem like much, but damn did it feel so hectic — maybe because it's my first time working on a real software project :)

After the setup, I finally started writing the core code. Right now it's just the foundation and an outline that I've set out to achieve.

The journey begins!

---

## DEVLOG #2

# The First Error and an Outline

*Session time: 57m 42s*

---

Ran into the first error — phewww!

Honestly, had it coming — things were going way too smoothly for a rather newbie like me.

I was working on the account system and trying to get the signup and login flow working when I hit the error. I also created a basic outline of flow (in my mind).

Spent a good chunk of time staring, retracing my code, and trying to figure out what exactly I had written wrong. But yes — I'm on it!

---

## DEVLOG #3

# The Plan Takes Shape

*Session time: 39m 40s*

---

Errors fixed and a basic outline has been made (this time on paper)!

I've also spent some time thinking about how I'm actually gonna build this thing. I'll have to make a SQL database (for account credentials and encryption details) and link it to my Python code.

And since these are journals — security and privacy matter a lot. My whole idea is that the actual journal entry is never stored. Only the encryption sequence and logic is saved. That way, even if someone got access to the database, they wouldn't find the journal itself.

The plan: 1. Finish the Python main code 2. Get encryption working properly 3. Add database support 4. Convert it into a website

I'm gonna complete the whole Python side tomorrow — and then hopefully convert it into a website!

---

## DEVLOG #4

# Still on the DB Thing

*Session time: 58m 33s*

---

Hehe — my high school Python-SQL interface notes are coming in handy!

When I was studying this stuff, I honestly didn't think I'd ever use most of it because it seemed too basic. Turns out I was wrong.

Since Privournal needs a database for account credentials and all that, I've started revising my old notes and figuring out how everything fits together.

Let's see how this goes!

---

## DEVLOG #5

# Database Established!

*Session time: 28m 5s*

---

Now the basic outline has to be perfected, with each and every thing being functional.

After spending some time figuring out SQL, databases, connections, and all that stuff, I finally managed to get Python talking to the database successfully.

Seeing actual records being fetched felt pretty satisfying, not gonna lie.

Current goals: - Add features and make everything functional - Connect everything together - Improve the overall structure

On the way to do that!

---

## DEVLOG #6

# Advanced Encryption!

*Session time: 1h 4m 42s*

---

Pheww — this part took some effort!

Today I worked on the main encryption logic for Advanced Encryption, and after a lot of thinking, coding, testing, and rethinking, I finally got the foundation for it down.

This is one of the core features of Privournal, so it feels really good seeing it slowly come together.

That said, this is only a small win. The real challenge is gonna be the 'Swiption' feature. Bet Swiption is gonna take half my brain cells.

---

## DEVLOG #7

# Almost There!

*Session time: 1h 42m 55s*

---

Only the Swiption feature remains!

I've also been thinking about adding another cool feature where covers are randomly chosen using Python's random module — it would be added inside Custom Encryption.

(Also, sorry — I sometimes referred to Custom Encryption as Advanced Encryption. I'll be consistent with names from now on.)

Once Swiption is done, the Python version should basically be mostly complete — and then it'll be time for the next step: getting all this on a website. That part honestly scares me a lot because I really don't know how to make websites. But I guess that's part of building a project — you learn whatever you need as you go.

Hopefully tomorrow the code will be done and I'll be one step closer to completing Privournal!

---

## DEVLOG #8

# The Eights

*Session time: 1h 8m 11s*

---

This is Privournal's 8th devlog, and this one feels kind of special — my birthday falls on an 8th too, and I just love the number 8.

Anyways, I was actually supposed to be done with Swiption by now, but I couldn't help myself and started working on the Random Encryption feature. The idea: why not make Random Encryption available for both Advanced Encryption AND Swiption? It would make the whole encryption process super fast for users who don't want to manually map each letter.

So I'm onto that — then I'll immediately look into Swiption.

I've also started mapping out the overall visual flow of Privournal because it was getting a bit confusing. I'm thinking of wrapping this up in about 12 devlogs total, then launching.

Thanks for reading!

---

## DEVLOG #9

# This Was Unexpected...

*Session time: 1h 11m 1s*

---

Turns out there were some issues in the decryption function. It completely ignored spaces while decrypting — which definitely wasn't ideal.

I've fixed that though.

I also remade the database tables with much more info: date of encryption, email (for forgot-password purposes), and more.

The code was also getting very long — over 400 lines. So I've started splitting things into different files to keep everything organized: DB code has its own file, big functions like AdvEn() have their own file, and the same goes for Swiption and Random Encryption logic.

Target: finish the code tonight. Thanks for reading!

---

## DEVLOG #10

# A Slight Setback

Session time: 4h 4m 45s

---

This is going to be the biggest devlog — and honestly a bit of a disappointing one.

My brother pointed out a flaw I completely missed. Say there's a word 'am' in a journal — 'a's cover is 1 and 'm's cover is 27, so 'am' encrypts to 127. But since there's no separator, the decryption code will traverse 1, then 2, then 7 separately. Major flaw. Fixing it by adding a trailing space after each cover.

Also noticed that in Advanced Encryption, users might accidentally repeat a cover for two different letters, making decryption ambiguous. Added a list to track used covers.

The code is getting complex — 600+ lines — so I carefully went through each function and made a proper flowchart to map everything out. It helps a lot.

Still showing 21 errors after all that — plus Swiption is still left. But one step at a time. See you in the next devlog!

---

## DEVLOG #11

# First Successful Test!

Session time: 3h 7m 43s

---

Let's go!!!

First successful test done!

The database fetching works, the encryption flow works — it wasn't working earlier and I spent an hour finding errors, running it again and again, and rewriting portions. My Advanced Encryption code was all messed up.

Some of the errors were genuinely silly: I had written `input("Enter cover for", x)` and couldn't figure out why it crashed — turns out `input()` only accepts one argument. A lot of other bugs were variable name confusion, functions not being global, and so on.

But yeah — one at a time I fixed them. Now only code refinement and a few more bugs remain.

I thought it'd wrap up in 10 devlogs, but it looks like the 12th will be the last. Hey — more the better, right? See you in the next devlog!

---

## DEVLOG #12

# IT WORKS!

*Session time: 1h 34m 26s*

---

FINALLY! After an hour of debugging — EVERYTHING WORKS!

- Encryption
- Decryption
- Uploading to & fetching from the database
- Sign Up
- Login

The only things not fully working yet are Swiption and Random Mode — honestly that's on me. I got way too focused on making everything else work first.

I had to improvise whole sections of the decryption logic. Some errors were genuinely silly: all my elif blocks somehow had the same condition (classic copy-paste disaster), and my traversal variable i was a string that I kept accidentally treating as an int.

One of the weirdest bugs: after decryption finally worked, the output had way too many extra spaces between words. After staring at it, I realized the number of spaces exactly matched the number of unique letters in the text. The loop was continuing even after finding a match — a few break statements later, everything worked.

On the 13th devlog, the project will be done done. See you in the next one!

---

## DEVLOG #13

# IT'S DONE DONE! The Final Chapter.

*Total time: 16h+*

---

FINALLY. After two intense days and more than 16 hours of grinding — Privournal is officially complete, polished, tested, and ready to ship!

I thought Devlog #11 was the 'everything works' moment, but the final stretch turned out to be the real work. I went back through literally every function, fixed dozens of lingering bugs, cleaned up the entire code, updated the README more than 10 times, dealt with PyPI packaging and GitHub releases, and recorded asciinema demos (which took hours — I kept getting bugs and trying to

make the perfect demo recording).

## MAJOR CHANGE: MySQL -> SQLite

Originally Privournal used MySQL with mysql-connector and python-dotenv so users could create accounts and store encryption keys in a central database. Later I realized a shared database requires a server to stay online — adding hosting, maintenance, and privacy concerns. Since Privournal's goal is to work completely offline, I redesigned it to use a local SQLite database instead. This made it easier to install, fully offline, more private, and much simpler to ship.

## DAY 1 — Bugs & Errors

I did a complete code audit — function by function through MainCode.py. Key fixes: - Account system: signup/login were flaky; fetchall() returned empty, user\_id wasn't linking, missing commit() calls, duplicate email handling was broken. - Decryption overhaul: that infamous 'extra spaces' bug (spaces = unique letter count) — finally killed it with proper break statements and redesigned De() logic. - JSON key loading for Advanced Encryption — handled JSONDecodeError and ensured dict reconstruction works every time. - Variable & scope issues: globals everywhere, copy-paste elif disasters, string vs int iterator bugs — all cleaned up. - Randomised Mode: fully functional and integrated smoothly with Advanced Encryption.

## DAY 2 — Swiption, Polishing, Packaging & Shipping

Swiption and its decryption counterpart SwipDe() were the absolute beasts of this project. The dynamic rotation logic with 'life' values and multi-dict tracking nearly broke my brain. At the end I realized I had massively over-complicated it — since each cover uniquely corresponds to a letter, I could just search through the dicts directly. SwipDe() was even worse: nested loops, matching stripped covers across rotations, fetching life values from the DB. I rewrote the traversal logic three times. Half the confusion was not knowing that JSON works for lists of multiple dicts. I ended up making a dedicated Swiption table in the DB just to keep things clean.

Swiption is now functional and feels like the standout feature I always wanted — my original idea for stronger encryption.

PyPI Packaging took way longer than expected: fixing setup and toml issues, making the code launch cleanly after pip install, testing in a fresh virtual environment.

GitHub: pushed final commits, cleaned the repo, added README images and demo links. Survived a scary moment where I thought a push had corrupted something.

Final Testing: ran full tests with and without account, Swiption + Random mode, edge cases (special characters, numbers, empty journals). Recorded a fresh asciinema demo and planned a YouTube commercial-style video.

Privournal is now a real, usable command-line journal encryptor.



Thanks to my brother (for catching those early logic flaws), GPT (for quick syntax rescues), and a big thanks to myself for pushing through all the silly bugs.

To install: `pip install privournal`

This journey was exhausting — but damn did it teach me something exams failed to: to work for something relentlessly with all your heart.

Thank you for reading all 13 devlogs. See you in the next project!

**PROJECT COMPLETE · READY TO SHIP ✦**

---